

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO CEARÁ

CAMPUS TIANGUÁ

DISCIPLINA: SISTEMAS DISTRIBUÍDOS

PROFESSOR: NÉCIO DE LIMA VERAS

RELATÓRIO TÉCNICO

Atividade sobre Processos e Threads em Java

José Leandro Costa Pereira

Tianguá – CE
2026

1. Identificação

Campo	Informação
Aluno(a)	JOSÉ LEANDRO COSTA PEREIRA
Matrícula	20211165000128
Curso	Ciências a Computação
Disciplina	Sistemas Distribuídos
Professor	Nécio de Lima Veras
Instituição	IFCE – Campus Tianguá
Atividade	Processos e Threads (prática individual)
Data de entrega	28 / 08 / 2026

2. Metodologia e arcabouço tecnológico

2.1 Ferramentas utilizadas

Item	Versão / Descrição
Linguagem	Java (sem bibliotecas externas, apenas java.lang)
JDK	OpenJDK 21.0.12 (Homebrew build), HotSpot 64-Bit Server VM
Sistema operacional	macOS 26.3 (Darwin 25.3.0)
Hardware	Apple M1, 8 núcleos, arquitetura arm64
Editor / IDE	Visual Studio Code
Compilação e execução	javac / java via terminal (script run.sh)
Controle de versão	Git + GitHub

2.2 Estratégia de implementação

Como as questões pedem modificações progressivas nas mesmas classes (Racer, Race, Deposito, Consumidor), optou-se por criar um pacote Java para cada etapa, preservando cada versão do código. Assim é possível executar e comparar qualquer item de forma independente:

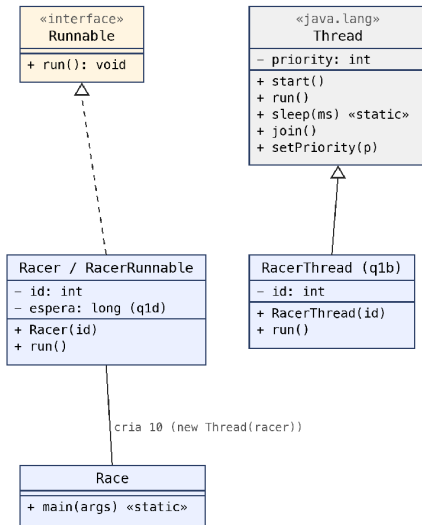
Pacote	Conteúdo
q1b	RacerThread (extends Thread), RacerRunnable (implements Runnable) e Main
q1c	Racer com while(true) e Race criando 10 racers
q1d	Racer com Thread.sleep(100) a cada impressão
q1e	Race com setPriority(i) (prioridade de 1 a 10)
q1f	Racer com 1000 impressões e Race usando join() (ímpares antes dos pares)
q2a	Deposito do enunciado + Produtor e Consumidor (sem controle)
q2c	Deposito com retirar() condicionado a itens > 0 (synchronized)
q3	Consumidor que espera 200 ms e tenta novamente quando o depósito está vazio

Para as questões com laço infinito, cada execução foi mantida por aproximadamente 2 segundos e então encerrada, com a saída redirecionada para arquivo (pasta saidas/). Um pequeno script de análise contou, para cada execução, a ordem da primeira impressão de cada

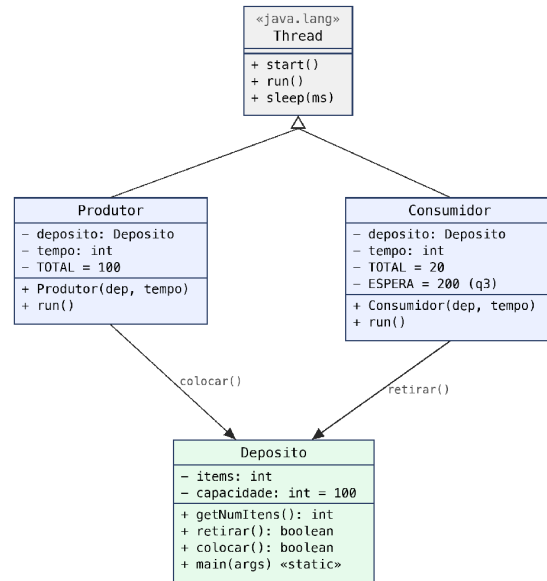
racer, a quantidade de impressões por racer e o tamanho médio das "rajadas" (sequências consecutivas de linhas do mesmo racer). Esses números embasam as análises das seções seguintes.

2.3 Diagrama de classes

Questão 1 – Racer / Race



Questões 2 e 3 – Produtor / Consumidor



Legenda: triângulo vazio = herança (contínua) / implementação (tracejada); seta cheia = uso. Na q2c/q3, retirar() e colocar() são synchronized.

Figura 1 – Diagrama de classes das Questões 1, 2 e 3

2.4 Métodos implementados

Classe	Método	Responsabilidade
Racer / RacerThread	run()	Laço de impressão "Racer i – imprimindo" (infinito ou 1000 vezes)
Race	main()	Cria os 10 racers, define prioridades (q1e) e sincroniza com join() (q1f)
Deposito	colocar()	Incrementa o número de caixas
Deposito	retirar()	Decrementa (q2a) ou decrementa somente se itens > 0 e retorna sucesso (q2c/q3)
Deposito	getNumItens()	Retorna a quantidade atual de caixas
Produtor	run()	Coloca 100 caixas, dormindo 'tempo' ms entre produções
Consumidor	run()	Retira 20 caixas, dormindo 'tempo' ms entre consumos; na q3 re-tenta a cada 200 ms

2.5 Dificuldades enfrentadas e soluções

- **Volume de saída com while(true):** em 2 segundos os 10 racers geram mais de 100 mil linhas, impossíveis de analisar no console. Solução: redirecionar a saída para arquivo e analisar os dados com um script (contagem por racer, ordem de largada e tamanho das rajadas).
- **Evolução das classes entre itens:** cada item altera o anterior. Solução: um pacote por item, mantendo todas as versões compiláveis e executáveis.
- **Identificar os consumidores na saída:** o construtor pedido recebe apenas (deposito, tempo). Solução: um contador estático que dá nome à thread ("Consumidor 1" a "Consumidor 5") via super(nome).
- **Resultado incorreto no depósito (q2a):** o estoque final não fechava em zero. A causa é a condição de corrida em `items = getNumItens() ± 1`, resolvida na q2c com métodos `synchronized`.
- **Ambiente:** a máquina não possuía JDK instalado; foi instalado o OpenJDK 21 via Homebrew e claude code para preparar o ambiente.

2.6 Repositório

Códigos completos: [Logx10/SistemasDistribuidos: Atividade de sistemas distribuidos-Threads](#)

Para executar: `./run.sh <pacote.Classe>` — por exemplo, `./run.sh q1f.Race` ou `./run.sh q2a.Deposito`.

3. Desenvolvimento

Questão 1

1a) Diferenças entre estender Thread e implementar Runnable

Descrição: explicar as diferenças, vantagens e desvantagens das duas formas de criação de threads em Java.

Em Java, uma thread é sempre representada por um objeto da classe `java.lang.Thread`; o código que ela executa fica no método `run()`. Há duas formas básicas de fornecer esse código:

- **Estender Thread:** a própria classe é uma `Thread` e sobrescreve `run()`. Basta instanciar e chamar `start()`.
- **Implementar Runnable:** a classe apenas descreve a tarefa (método `run()`); o objeto é passado ao construtor de `Thread`, que é quem de fato executa: `new Thread(new Racer(1)).start()`.

Critério	extends Thread	implements Runnable
Herança	Consome a única herança permitida em Java	A classe continua livre para estender outra
Separação de responsabilidades	Mistura "a tarefa" com "o mecanismo de execução"	Tarefa separada da thread (baixo acoplamento)
Reuso	Cada objeto só pode ser executado uma vez, como aquela thread	O mesmo Runnable pode ser executado por várias threads ou por um <code>ExecutorService</code> / pool
Acesso aos métodos de Thread	Direto (<code>getName()</code> , <code>setPriority()</code> , etc.)	Via <code>Thread.currentThread()</code> ou pelo objeto <code>Thread</code> criado
Simplicidade	Um pouco menos de código	Uma linha a mais (criar o objeto <code>Thread</code>); compatível com lambdas
Recomendação	Casos simples ou quando se quer customizar o comportamento da thread	Abordagem preferida na prática

Em resumo: estender `Thread` é mais direto, mas "gasta" a herança e acopla a tarefa à thread. Implementar `Runnable` é mais flexível, favorece composição sobre herança e é a forma usada pelas APIs modernas de concorrência (`ExecutorService`, `CompletableFuture`, `virtual threads`). Em ambos os casos, é `start()` — e não `run()` — que cria a nova linha de execução; chamar `run()` diretamente apenas executa o método na thread atual.

1b) Classe Racer nas duas formas

Descrição: criar a classe `Racer` com um `while(true)` que imprime "`Racer i – imprimindo`", recebendo `i` no construtor, usando as duas formas de criação e instanciação.

Explicação: o identificador é guardado em um atributo final recebido pelo construtor. Na forma 1 a classe estende `Thread` e é iniciada diretamente; na forma 2 implementa `Runnable` e é embrulhada por um objeto `Thread`.

Arquivo: `src/q1b/RacerThread.java`

```
package q1b;
```

```
/**
```

```

* Questão 1b - Forma 1: estendendo a classe Thread.
*/
public class RacerThread extends Thread {
    private final int id;

    public RacerThread(int id) {
        this.id = id;
    }

    @Override
    public void run() {
        while (true) {
            System.out.println("Racer " + id + " - imprimindo");
        }
    }
}

```

Arquivo: src/q1b/RacerRunnable.java

```

package q1b;

/**
 * Questão 1b - Forma 2: implementando a interface Runnable.
 */
public class RacerRunnable implements Runnable {
    private final int id;

    public RacerRunnable(int id) {
        this.id = id;
    }

    @Override
    public void run() {
        while (true) {
            System.out.println("Racer " + id + " - imprimindo");
        }
    }
}

```

Arquivo: src/q1b/Main.java

```

package q1b;

/**
 * Questão 1b - Demonstra as duas formas de criação e instanciação.
 */
public class Main {
    public static void main(String[] args) {
        // Forma 1: a própria classe é uma Thread
        RacerThread r1 = new RacerThread(1);
        r1.start();

        // Forma 2: o Runnable é entregue a um objeto Thread
        Thread r2 = new Thread(new RacerRunnable(2));
        r2.start();
    }
}

```

```
Terminal - java q1b

$ java -cp out q1b.Main
... (linha 5460)
Racer 1 - imprimindo
Racer 2 - imprimindo
Racer 2 - imprimindo
Racer 2 - imprimindo
Racer 2 - imprimindo
Racer 1 - imprimindo
Racer 1 - imprimindo
Racer 1 - imprimindo
Racer 1 - imprimindo
Racer 1 - imprimindo
Racer 2 - imprimindo
Racer 2 - imprimindo
Racer 2 - imprimindo
Racer 2 - imprimindo
Racer 1 - imprimindo
Racer 1 - imprimindo
Racer 1 - imprimindo
Racer 1 - imprimindo
Racer 2 - imprimindo
Racer 1 - imprimindo
...
^C
# ordem da 1a impressao: [1, 2]
# total de linhas em ~2s: 170671
# prints por racer: R1=85983 R2=84688
```

Figura 2 – Execução de q1b.Main – Racer 1 (Thread) e Racer 2 (Runnable) executando concorrentemente

Análise: as duas formas produzem exatamente o mesmo comportamento em execução. Em 2 s foram impressas cerca de 170 mil linhas, divididas quase igualmente entre os dois racers (≈ 86 mil cada). Na terceira execução o Racer 2 imprimiu primeiro, apesar de ter sido iniciado depois, mostrando que a ordem de start() não garante a ordem de execução.

1c) Classe Race com 10 racers

Descrição: criar a classe Race que instancia 10 racers (1 a 10), descrever o comportamento dos prints e comentar os resultados de diferentes execuções.

Explicação: cada racer é uma thread independente disputando a CPU e, principalmente, o recurso compartilhado System.out. Quem decide qual thread executa a cada instante é o escalonador do sistema operacional (as threads da JVM HotSpot são threads nativas), de forma preemptiva e não determinística.

Arquivo: src/q1c/Racer.java

```
package q1c;
```

```
public class Racer implements Runnable {
    private final int id;

    public Racer(int id) {
        this.id = id;
    }

    @Override
    public void run() {
        while (true) {
            System.out.println("Racer " + id + " - imprimindo");
        }
    }
}
```

Arquivo: src/q1c/Race.java

```
package q1c;

/**
 * Questão 1c - Cria e inicia 10 racers (1 a 10).
 */
public class Race {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            new Thread(new Racer(i)).start();
        }
    }
}
```

```
Terminal - java q1c_run1

$ java -cp out q1c.Race
Racer 1 - imprimindo
Racer 1 - imprimindo
Racer 1 - imprimindo
Racer 1 - imprimindo
Racer 1 - imprimindo
Racer 1 - imprimindo
... (linha 22043)
Racer 8 - imprimindo
Racer 8 - imprimindo
Racer 8 - imprimindo
Racer 8 - imprimindo
Racer 8 - imprimindo
Racer 8 - imprimindo
Racer 8 - imprimindo
Racer 8 - imprimindo
Racer 8 - imprimindo
Racer 8 - imprimindo
Racer 8 - imprimindo
Racer 8 - imprimindo
Racer 9 - imprimindo
Racer 7 - imprimindo
Racer 4 - imprimindo
Racer 7 - imprimindo
Racer 9 - imprimindo
Racer 7 - imprimindo
Racer 9 - imprimindo
Racer 9 - imprimindo
Racer 9 - imprimindo
Racer 9 - imprimindo
Racer 9 - imprimindo
...
^C
# ordem da 1a impressao: [1, 3, 4, 8, 10, 5, 2, 7, 9, 6]
# total de linhas em ~2s: 80132
# prints por racer: R1=8886 R2=10876 R3=9335 R4=7039 R5=6114
# R6=6642 R7=7319 R8=9371 R9=6989 R10=7561
```

Figura 3 – q1c – execução 1

```
Terminal - java q1c_run2

$ java -cp out q1c.Race
Racer 8 - imprimindo
Racer 8 - imprimindo
Racer 8 - imprimindo
Racer 8 - imprimindo
Racer 8 - imprimindo
Racer 8 - imprimindo
... (linha 56)
Racer 6 - imprimindo
Racer 8 - imprimindo
Racer 7 - imprimindo
Racer 9 - imprimindo
Racer 5 - imprimindo
Racer 5 - imprimindo
Racer 5 - imprimindo
Racer 5 - imprimindo
Racer 5 - imprimindo
Racer 5 - imprimindo
Racer 5 - imprimindo
Racer 5 - imprimindo
Racer 5 - imprimindo
Racer 5 - imprimindo
Racer 5 - imprimindo
Racer 10 - imprimindo
Racer 10 - imprimindo
Racer 4 - imprimindo
Racer 2 - imprimindo
Racer 2 - imprimindo
Racer 3 - imprimindo
Racer 1 - imprimindo
...
^C
# ordem da 1a impressao: [8, 7, 9, 5, 10, 4, 2, 3, 1, 6]
# total de linhas em ~2s: 115123
# prints por racer: R1=9080 R2=11079 R3=15557 R4=11046
# R5=6966 R6=11475 R7=10937 R8=15962 R9=11914 R10=11107
```

Figura 4 – q1c – execução 2

```
Terminal - java q1c_run3

$ java -cp out q1c.Race
Racer 3 - imprimindo
Racer 3 - imprimindo
Racer 3 - imprimindo
Racer 3 - imprimindo
Racer 3 - imprimindo
Racer 2 - imprimindo
... (linha 7)
Racer 2 - imprimindo
Racer 6 - imprimindo
Racer 8 - imprimindo
Racer 7 - imprimindo
Racer 4 - imprimindo
Racer 10 - imprimindo
Racer 10 - imprimindo
Racer 10 - imprimindo
Racer 10 - imprimindo
Racer 10 - imprimindo
Racer 10 - imprimindo
Racer 10 - imprimindo
Racer 9 - imprimindo
Racer 3 - imprimindo
Racer 5 - imprimindo
Racer 1 - imprimindo
Racer 2 - imprimindo
Racer 6 - imprimindo
Racer 8 - imprimindo
Racer 7 - imprimindo
Racer 4 - imprimindo
...
^C
# ordem da 1a impressao: [3, 2, 6, 8, 7, 4, 10, 9, 5, 1]
# total de linhas em ~2s: 112863
# prints por racer: R1=10198 R2=16878 R3=7786 R4=9727
# R5=11934 R6=14123 R7=12557 R8=10726 R9=10081 R10=8853
```

Figura 5 – q1c – execução 3

Execução	Ordem da 1ª impressão	Mais impressões	Menos impressões	Rajada média
1	1, 3, 4, 8, 10, 5, 2, 7, 9, 6	Racer 2 (10.876)	Racer 5 (6.114)	43 linhas
2	8, 7, 9, 5, 10, 4, 2, 3, 1, 6	Racer 8 (15.962)	Racer 5 (6.966)	46 linhas
3	3, 2, 6, 8, 7, 4, 10, 9, 5, 1	Racer 2 (16.878)	Racer 3 (7.786)	65 linhas

Análise dos resultados:

- Os prints não se alternam de forma ordenada: aparecem em rajadas — um racer imprime dezenas (às vezes milhares, até ~3.000) de linhas seguidas antes de outro conseguir imprimir. Isso ocorre porque `println()` é sincronizado internamente: a thread que detém o lock do `System.out` tende a readquiri-lo várias vezes antes de o escalonador trocar de thread.
- A ordem de largada muda a cada execução, mesmo com os racers sendo iniciados na ordem 1 → 10. `start()` apenas torna a thread elegível para execução; quando ela realmente começa depende do escalonador.
- O "vencedor" (quem mais imprimiu no intervalo) também muda: Racer 2, Racer 8 e novamente Racer 2, com diferenças de até 2× entre o mais rápido e o mais lento. Não há como prever o resultado da corrida — comportamento não determinístico típico de programas concorrentes.

1d) Racers com sleep

Descrição: adicionar um tempo de espera com `sleep` nos racers e explicar o que houve com o sistema.

Explicação: `Thread.sleep(ms)` coloca a thread no estado `TIMED_WAITING`, liberando a CPU (e o lock de `System.out`) por aquele intervalo. Foi usado `sleep` de 100 ms após cada impressão. Como `sleep` pode lançar `InterruptedException`, ela é tratada restaurando o status de interrupção e encerrando a thread.

Arquivo: `src/q1d/Racer.java`

```
package q1d;

public class Racer implements Runnable {
    private final int id;
    private final long espera;

    public Racer(int id, long espera) {
        this.id = id;
        this.espera = espera;
    }

    @Override
    public void run() {
        while (true) {
            System.out.println("Racer " + id + " - imprimindo");
            try {
                Thread.sleep(espera); // libera a CPU por 'espera' ms
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
    }
}
```

Arquivo: `src/q1d/Race.java`

```
package q1d;
```

```
/**
 * Questão 1d - Racers com tempo de espera (sleep) de 100 ms.
 */
public class Race {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            new Thread(new Racer(i, 100)).start();
        }
    }
}
```

```
Terminal - java q1d_run1

$ java -cp out q1d.Race
Racer 8 - imprimindo
Racer 1 - imprimindo
Racer 2 - imprimindo
Racer 10 - imprimindo
Racer 5 - imprimindo
Racer 4 - imprimindo
Racer 3 - imprimindo
Racer 7 - imprimindo
Racer 6 - imprimindo
Racer 9 - imprimindo
Racer 8 - imprimindo
Racer 4 - imprimindo
Racer 5 - imprimindo
Racer 10 - imprimindo
Racer 1 - imprimindo
Racer 3 - imprimindo
Racer 2 - imprimindo
Racer 7 - imprimindo
Racer 6 - imprimindo
Racer 9 - imprimindo
Racer 8 - imprimindo
Racer 4 - imprimindo
Racer 3 - imprimindo
Racer 1 - imprimindo
Racer 5 - imprimindo
Racer 2 - imprimindo
Racer 10 - imprimindo
Racer 7 - imprimindo
Racer 6 - imprimindo
Racer 9 - imprimindo
...
^C
# ordem da 1a impressao: [8, 1, 2, 10, 5, 4, 3, 7, 6, 9]
# total de linhas em ~2s: 180
# prints por racer: R1=18 R2=18 R3=18 R4=18 R5=18 R6=18
# R7=18 R8=18 R9=18 R10=18
```

Figura 6 – q1d – execução 1 (sleep de 100 ms)

```
Terminal - java q1d_run2

$ java -cp out q1d.Race
Racer 8 - imprimindo
Racer 9 - imprimindo
Racer 3 - imprimindo
Racer 7 - imprimindo
Racer 2 - imprimindo
Racer 4 - imprimindo
Racer 5 - imprimindo
Racer 6 - imprimindo
Racer 10 - imprimindo
Racer 1 - imprimindo
Racer 7 - imprimindo
Racer 8 - imprimindo
Racer 9 - imprimindo
Racer 3 - imprimindo
Racer 10 - imprimindo
Racer 4 - imprimindo
Racer 5 - imprimindo
Racer 2 - imprimindo
Racer 6 - imprimindo
Racer 1 - imprimindo
Racer 7 - imprimindo
Racer 8 - imprimindo
Racer 9 - imprimindo
Racer 3 - imprimindo
Racer 1 - imprimindo
Racer 5 - imprimindo
Racer 10 - imprimindo
Racer 4 - imprimindo
Racer 2 - imprimindo
Racer 6 - imprimindo
...
^C
# ordem da 1a impressao: [8, 9, 3, 7, 2, 4, 5, 6, 10, 1]
# total de linhas em ~2s: 186
# prints por racer: R1=18 R2=19 R3=18 R4=18 R5=19 R6=19
#   R7=19 R8=19 R9=19 R10=18
```

Figura 7 – q1d – execução 2 (sleep de 100 ms)

Análise:

- O volume de saída caiu de ~100 mil linhas para ~180 linhas em 2 s — cerca de 18 impressões por racer ($\approx 2000 \text{ ms} / 100 \text{ ms}$), e o uso de CPU caiu praticamente a zero, pois as threads passam quase todo o tempo dormindo.

- As rajadas desapareceram: nenhum racer imprimiu duas vezes seguidas (rajada média = 1,0). A saída passa a ser organizada em "rodadas": cada racer imprime uma vez a cada 100 ms.
- A distribuição ficou justa: todos os racers imprimiram a mesma quantidade (± 1).
- A ordem dentro de cada rodada continua variando, pois todas as threads acordam praticamente no mesmo instante e disputam a CPU — o sleep reduz a disputa, mas não elimina o não determinismo.

1e) Prioridades com setPriority

Descrição: usar setPriority para definir as condições de corrida e verificar se houve mudança na execução.

Explicação: cada thread Java possui prioridade de 1 (MIN_PRIORITY) a 10 (MAX_PRIORITY), com padrão 5. Foi atribuída ao Racer i a prioridade i, ou seja, Racer 10 com prioridade máxima e Racer 1 com mínima. Esperava-se que os racers de maior prioridade imprimissem mais.

Arquivo: src/q1e/Race.java (Racer igual ao da q1c)

```
package q1e;

/**
 * Questão 1e - Define prioridades diferentes para os racers.
 * Racer 10 recebe a prioridade máxima (10), Racer 1 a mínima (1)
 * e os demais recebem prioridade igual ao seu identificador.
 */
public class Race {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            Thread t = new Thread(new Racer(i));
            t.setPriority(i); // Thread.MIN_PRIORITY = 1 ... Thread.MAX_PRIORITY = 10
            t.start();
        }
    }
}
```

```
Terminal - java q1e_run1

$ java -cp out q1e.Race
Racer 6 - imprimindo
Racer 6 - imprimindo
Racer 6 - imprimindo
Racer 6 - imprimindo
Racer 6 - imprimindo
Racer 6 - imprimindo
... (linha 497)
Racer 1 - imprimindo
Racer 1 - imprimindo
Racer 1 - imprimindo
Racer 1 - imprimindo
Racer 5 - imprimindo
Racer 9 - imprimindo
Racer 10 - imprimindo
Racer 10 - imprimindo
Racer 2 - imprimindo
Racer 8 - imprimindo
Racer 7 - imprimindo
Racer 7 - imprimindo
Racer 7 - imprimindo
Racer 3 - imprimindo
Racer 4 - imprimindo
Racer 6 - imprimindo
Racer 6 - imprimindo
Racer 1 - imprimindo
Racer 1 - imprimindo
Racer 5 - imprimindo
Racer 5 - imprimindo
Racer 5 - imprimindo
...
^C
# ordem da 1a impressao: [6, 1, 5, 10, 9, 2, 8, 7, 4, 3]
# total de linhas em ~2s: 147644
# prints por racer: R1=10630 R2=16292 R3=15236 R4=13988
# R5=19792 R6=13210 R7=19232 R8=14269 R9=13871 R10=11124
```

Figura 8 – q1e – execução 1 (prioridade = identificador)

```
Terminal - java q1e_run2

$ java -cp out q1e.Race
Racer 2 - imprimindo
Racer 2 - imprimindo
Racer 2 - imprimindo
Racer 2 - imprimindo
Racer 2 - imprimindo
Racer 2 - imprimindo
... (linha 406)
Racer 6 - imprimindo
Racer 4 - imprimindo
Racer 4 - imprimindo
Racer 9 - imprimindo
Racer 9 - imprimindo
Racer 7 - imprimindo
Racer 7 - imprimindo
Racer 2 - imprimindo
Racer 2 - imprimindo
Racer 10 - imprimindo
Racer 10 - imprimindo
Racer 3 - imprimindo
Racer 8 - imprimindo
Racer 5 - imprimindo
Racer 1 - imprimindo
Racer 1 - imprimindo
Racer 1 - imprimindo
Racer 1 - imprimindo
Racer 6 - imprimindo
Racer 4 - imprimindo
Racer 9 - imprimindo
Racer 7 - imprimindo
...
^C
# ordem da 1a impressao: [2, 3, 8, 10, 5, 1, 6, 4, 9, 7]
# total de linhas em ~2s: 120774
# prints por racer: R1=15745 R2=15055 R3=10079 R4=10231
# R5=9950 R6=10940 R7=11885 R8=10834 R9=12597 R10=13458
```

Figura 9 – q1e – execução 2 (prioridade = identificador)

Execução	Mais impressões	Racer 10 (prioridade 10)	Racer 1 (prioridade 1)
1	Racer 5 (19.792)	11.124	10.630
2	Racer 1 (15.745)	13.458	15.745
3	Racer 9 (13.395)	9.374	13.366

Análise: não houve mudança perceptível. Não existe correlação entre prioridade e quantidade de impressões — na execução 2 o racer de prioridade mínima foi justamente o que mais imprimiu. Os motivos são:

- Prioridade em Java é apenas uma sugestão ao escalonador; a especificação não garante nenhum efeito. Na JVM HotSpot em macOS e Linux as prioridades Java praticamente não são repassadas ao sistema operacional com a configuração padrão.
- A máquina tem 8 núcleos para 10 threads: quase todas executam em paralelo real, então há pouca disputa por CPU que o escalonador precisaria arbitrar por prioridade.
- O verdadeiro gargalo é o lock do System.out, e a aquisição de locks em Java não respeita prioridade.

Conclusão do item: a corretude de um programa concorrente nunca deve depender de prioridades de threads.

1f) 1000 impressões e join

Descrição: fazer o Racer imprimir apenas 1000 vezes e modificar Race para que os carros pares só iniciem quando os ímpares terminarem, usando join.

Explicação: t.join() bloqueia a thread que o chama (aqui, a main) até que a thread t termine. A main inicia os cinco ímpares, faz join em cada um e só então inicia os pares. Um segundo conjunto de joins garante que a mensagem final só apareça quando todos terminarem.

Arquivo: src/q1f/Racer.java

```
package q1f;

public class Racer implements Runnable {
    private final int id;

    public Racer(int id) {
        this.id = id;
    }

    @Override
    public void run() {
        for (int volta = 1; volta <= 1000; volta++) {
            System.out.println("Racer " + id + " - imprimindo (" + volta + ")");
        }
        System.out.println(">>> Racer " + id + " terminou a corrida");
    }
}
```

Arquivo: src/q1f/Race.java

```
package q1f;

import java.util.ArrayList;
import java.util.List;

/**
 * Questão 1f - Os carros pares só largam depois que todos os ímpares terminarem.
 */
public class Race {
    public static void main(String[] args) throws InterruptedException {
        List<Thread> impares = new ArrayList<>();
```

```
List<Thread> pares = new ArrayList<>();

for (int i = 1; i <= 10; i++) {
    Thread t = new Thread(new Racer(i));
    if (i % 2 != 0) impares.add(t); else pares.add(t);
}

System.out.println("=== Largada dos ímpares ===");
for (Thread t : impares) t.start();

// join: a thread main fica bloqueada até cada ímpar terminar
for (Thread t : impares) t.join();

System.out.println("=== Ímpares terminaram. Largada dos pares ===");
for (Thread t : pares) t.start();
for (Thread t : pares) t.join();

System.out.println("=== Corrida encerrada ===");
}
}
```

```
Terminal - java q1f_run1

$ java -cp out q1f.Race
=== Largada dos ímpares ===
Racer 9 - imprimindo (1)
Racer 9 - imprimindo (2)
Racer 9 - imprimindo (3)
Racer 9 - imprimindo (4)
...
Racer 9 - imprimindo (994)
Racer 9 - imprimindo (995)
Racer 9 - imprimindo (996)
Racer 9 - imprimindo (997)
Racer 9 - imprimindo (998)
Racer 9 - imprimindo (999)
Racer 9 - imprimindo (1000)
>>> Racer 9 terminou a corrida
=== Ímpares terminaram. Largada dos pares ===
Racer 2 - imprimindo (1)
Racer 2 - imprimindo (2)
Racer 2 - imprimindo (3)
Racer 2 - imprimindo (4)
Racer 2 - imprimindo (5)
...
Racer 8 - imprimindo (999)
Racer 8 - imprimindo (1000)
>>> Racer 8 terminou a corrida
=== Corrida encerrada ===

# marcos da execucao:
=== Largada dos ímpares ===
>>> Racer 5 terminou a corrida
>>> Racer 3 terminou a corrida
>>> Racer 7 terminou a corrida
>>> Racer 1 terminou a corrida
>>> Racer 9 terminou a corrida
=== Ímpares terminaram. Largada dos pares ===
>>> Racer 10 terminou a corrida
>>> Racer 2 terminou a corrida
>>> Racer 4 terminou a corrida
>>> Racer 6 terminou a corrida
>>> Racer 8 terminou a corrida
=== Corrida encerrada ===
```

Figura 10 – q1f – execução 1: transição entre ímpares e pares

```
Terminal - java q1f_run2

$ java -cp out q1f.Race
=== Largada dos ímpares ===
Racer 5 - imprimindo (1)
Racer 7 - imprimindo (1)
Racer 7 - imprimindo (2)
Racer 7 - imprimindo (3)
...
Racer 3 - imprimindo (994)
Racer 3 - imprimindo (995)
Racer 3 - imprimindo (996)
Racer 3 - imprimindo (997)
Racer 3 - imprimindo (998)
Racer 3 - imprimindo (999)
Racer 3 - imprimindo (1000)
>>> Racer 3 terminou a corrida
=== Ímpares terminaram. Largada dos pares ===
Racer 2 - imprimindo (1)
Racer 2 - imprimindo (2)
Racer 2 - imprimindo (3)
Racer 2 - imprimindo (4)
Racer 2 - imprimindo (5)
...
Racer 8 - imprimindo (999)
Racer 8 - imprimindo (1000)
>>> Racer 8 terminou a corrida
=== Corrida encerrada ===

# marcos da execucao:
=== Largada dos ímpares ===
>>> Racer 5 terminou a corrida
>>> Racer 7 terminou a corrida
>>> Racer 1 terminou a corrida
>>> Racer 9 terminou a corrida
>>> Racer 3 terminou a corrida
=== Ímpares terminaram. Largada dos pares ===
>>> Racer 10 terminou a corrida
>>> Racer 6 terminou a corrida
>>> Racer 2 terminou a corrida
>>> Racer 4 terminou a corrida
>>> Racer 8 terminou a corrida
=== Corrida encerrada ===
```

Figura 11 – q1f – execução 2

Execução	Ordem de chegada dos ímpares	Ordem de chegada dos pares
1	5, 3, 7, 1, 9	10, 2, 4, 6, 8

2	5, 7, 1, 9, 3	10, 6, 2, 4, 8
3	7, 1, 5, 9, 3	6, 2, 8, 10, 4

Análise: nas três execuções, as 5.000 linhas dos ímpares aparecem antes da largada dos pares (linha 5.007) e nenhum par imprimiu antes disso — verificado automaticamente nos arquivos de saída. O join garantiu uma ordem entre os grupos, mas dentro de cada grupo a corrida continua não determinística: a ordem de chegada mudou em todas as execuções. Com 1000 impressões o programa termina sozinho (10.013 linhas no total), ao contrário das versões com while(true).

Questão 2

Descrição: criar as classes Produtor e Consumidor, como threads, que aumentam e diminuem a variável items da classe Deposito. O Produtor produz 100 caixas com intervalo recebido no construtor; cada Consumidor consome 20 caixas com intervalo recebido no construtor. O main usa 1 produtor (50 ms) e 5 consumidores (150, 100, 150, 100 e 150 ms).

2a) Código das classes

Explicação: Produtor e Consumidor estendem Thread (o main do enunciado chama p.start() diretamente) e recebem a referência para o mesmo Deposito — que é, portanto, um recurso compartilhado entre 6 threads. Entre cada operação a thread dorme o tempo recebido, tornando a produção/consumo não contínua. A classe Deposito foi mantida exatamente como no enunciado.

Arquivo: src/q2a/Deposito.java

```
package q2a;

/**
 * Questão 2a - Classe Deposito conforme o enunciado (sem nenhum controle).
 */
public class Deposito {
    private int items = 0;
    private final int capacidade = 100;

    public int getNumItens() {
        return items;
    }

    public boolean retirar() {
        items = getNumItens() - 1;
        return true;
    }

    public boolean colocar() {
        items = getNumItens() + 1;
        return true;
    }

    public static void main(String[] args) {
        Deposito dep = new Deposito();
        Produtor p = new Produtor(dep, 50);
        Consumidor c1 = new Consumidor(dep, 150);
        Consumidor c2 = new Consumidor(dep, 100);
        Consumidor c3 = new Consumidor(dep, 150);
        Consumidor c4 = new Consumidor(dep, 100);
        Consumidor c5 = new Consumidor(dep, 150);
        //Startar o produtor
        p.start();
        //Startar os consumidores.
        c1.start(); c2.start(); c3.start();
        c4.start(); c5.start();
        System.out.println("Execucao do main da classe Deposito terminada");
    }
}
```

Arquivo: src/q2a/Produtor.java

```

package q2a;

/**
 * Produz 100 caixas, esperando 'tempo' ms entre cada produção.
 */
public class Produtor extends Thread {
    private static final int TOTAL = 100;
    private final Deposito deposito;
    private final int tempo;

    public Produtor(Deposito deposito, int tempo) {
        super("Produtor");
        this.deposito = deposito;
        this.tempo = tempo;
    }

    @Override
    public void run() {
        for (int i = 1; i <= TOTAL; i++) {
            deposito.colocar();
            System.out.println(getName() + " colocou a caixa " + i
                               + " | itens no deposito: " + deposito.getNumItens());
            try {
                Thread.sleep(tempo);
            } catch (InterruptedException e) {
                return;
            }
        }
        System.out.println(getName() + " terminou: produziu " + TOTAL + " caixas");
    }
}

```

Arquivo: src/q2a/Consumidor.java

```

package q2a;

/**
 * Consome 20 caixas, esperando 'tempo' ms entre cada consumo.
 */
public class Consumidor extends Thread {
    private static final int TOTAL = 20;
    private static int contador = 0;
    private final Deposito deposito;
    private final int tempo;

    public Consumidor(Deposito deposito, int tempo) {
        super("Consumidor " + (++contador));
        this.deposito = deposito;
        this.tempo = tempo;
    }

    @Override
    public void run() {
        for (int i = 1; i <= TOTAL; i++) {
            deposito.retirar();
            System.out.println(getName() + " retirou a caixa " + i
                               + " | itens no deposito: " + deposito.getNumItens());
            try {
                Thread.sleep(tempo);
            } catch (InterruptedException e) {
            }
        }
    }
}

```

```

        return;
    }
}
System.out.println(getName() + " terminou: consumiu " + TOTAL + " caixas");
}
}

```

```

Terminal - java q2a_run1

$ java -cp out q2a.Deposito
Execucao do main da classe Deposito terminada
Consumidor 5 retirou a caixa 1 | itens no deposito: -4
Consumidor 2 retirou a caixa 1 | itens no deposito: -1
Consumidor 3 retirou a caixa 1 | itens no deposito: -2
Consumidor 1 retirou a caixa 1 | itens no deposito: 0
Consumidor 4 retirou a caixa 1 | itens no deposito: -3
Produtor colocou a caixa 1 | itens no deposito: 1
Produtor colocou a caixa 2 | itens no deposito: -3
Consumidor 2 retirou a caixa 2 | itens no deposito: -4
Consumidor 4 retirou a caixa 2 | itens no deposito: -4
Produtor colocou a caixa 3 | itens no deposito: -3
Consumidor 3 retirou a caixa 2 | itens no deposito: -4
Consumidor 5 retirou a caixa 2 | itens no deposito: -5
Consumidor 1 retirou a caixa 2 | itens no deposito: -6
...
Execucao do main da classe Deposito terminada
Consumidor 4 terminou: consumiu 20 caixas
Consumidor 2 terminou: consumiu 20 caixas
Consumidor 3 terminou: consumiu 20 caixas
Consumidor 5 terminou: consumiu 20 caixas
Consumidor 1 terminou: consumiu 20 caixas
Produtor terminou: produziu 100 caixas
...
# ultimas linhas:
Produtor colocou a caixa 96 | itens no deposito: 0
Produtor colocou a caixa 97 | itens no deposito: 1
Produtor colocou a caixa 98 | itens no deposito: 2
Produtor colocou a caixa 99 | itens no deposito: 3
Produtor colocou a caixa 100 | itens no deposito: 4
Produtor terminou: produziu 100 caixas

```

Figura 12 – q2a – execução 1

```
Terminal - java q2a_run2

$ java -cp out q2a.Deposito
Execucao do main da classe Deposito terminada
Consumidor 3 retirou a caixa 1 | itens no deposito: -2
Consumidor 4 retirou a caixa 1 | itens no deposito: -3
Produtor colocou a caixa 1 | itens no deposito: 1
Consumidor 1 retirou a caixa 1 | itens no deposito: 0
Consumidor 2 retirou a caixa 1 | itens no deposito: -1
Consumidor 5 retirou a caixa 1 | itens no deposito: -4
Produtor colocou a caixa 2 | itens no deposito: -3
Consumidor 4 retirou a caixa 2 | itens no deposito: -4
Consumidor 2 retirou a caixa 2 | itens no deposito: -5
Produtor colocou a caixa 3 | itens no deposito: -4
Consumidor 3 retirou a caixa 2 | itens no deposito: -5
Consumidor 5 retirou a caixa 2 | itens no deposito: -6
Consumidor 1 retirou a caixa 2 | itens no deposito: -7
...
Execucao do main da classe Deposito terminada
Consumidor 2 terminou: consumiu 20 caixas
Consumidor 4 terminou: consumiu 20 caixas
Consumidor 3 terminou: consumiu 20 caixas
Consumidor 5 terminou: consumiu 20 caixas
Consumidor 1 terminou: consumiu 20 caixas
Produtor terminou: produziu 100 caixas
...
# ultimas linhas:
Produtor colocou a caixa 96 | itens no deposito: -3
Produtor colocou a caixa 97 | itens no deposito: -2
Produtor colocou a caixa 98 | itens no deposito: -1
Produtor colocou a caixa 99 | itens no deposito: 0
Produtor colocou a caixa 100 | itens no deposito: 1
Produtor terminou: produziu 100 caixas
```

Figura 13 – q2a – execução 2

2b) Última mensagem e por quê

Última mensagem: "Produtor terminou: produziu 100 caixas" (nas duas execuções).

Por quê:

- A mensagem "Execucao do main da classe Deposito terminada" é uma das primeiras: a main apenas inicia as threads e termina. A JVM continua em execução porque as threads criadas não são daemon.
- Os tempos explicam a ordem de término: consumidores de 100 ms terminam em ~2 s (20×100 ms), os de 150 ms em ~3 s, enquanto o produtor leva ~5 s (100×50 ms). Por isso o produtor é sempre o último.

- O ritmo de consumo ($2 \times 10 + 3 \times 6,7 \approx 40$ caixas/s) é o dobro do de produção (20 caixas/s). Como retirar() não verifica nada, o estoque fica negativo, chegando a -43 e -44 caixas — uma situação impossível no mundo real.
- O estoque final deveria ser 0 (100 produzidas – 100 consumidas), mas foi 4 na execução 1 e 1 na execução 2. Isso é uma condição de corrida (race condition): items = getNumItens() – 1 não é atômico (ler, calcular, escrever). Quando duas threads leem o mesmo valor antes de qualquer uma escrever, uma das atualizações é perdida (lost update). Também se vê na saída valores fora de ordem, como "Produtor colocou a caixa 2 | itens: -3" logo após "itens: 1", porque a leitura para impressão ocorre depois e outras threads já alteraram o valor.

2c) Retirar somente se items > 0

Explicação: retirar() passa a verificar a pré-condição items > 0 e retorna false quando o depósito está vazio. Para que o teste e o decremento sejam uma operação única e indivisível, os métodos foram marcados como synchronized: apenas uma thread por vez executa código sincronizado no mesmo objeto Deposito (exclusão mútua pelo monitor do objeto). Sem isso, dois consumidores poderiam ver items == 1 ao mesmo tempo e ambos retirar, voltando a gerar valores negativos. O Consumidor passa a contar quantas caixas realmente conseguiu retirar nas suas 20 tentativas.

Arquivo: src/q2c/Deposito.java

```
package q2c;

/**
 * Questão 2c - Só permite retirar se houver itens (items > 0).
 * Os métodos são synchronized para que o teste "items > 0" e o decremento
 * aconteçam de forma atômica (evita condição de corrida entre consumidores).
 */
public class Deposito {
    private int items = 0;
    private final int capacidade = 100;

    public synchronized int getNumItens() {
        return items;
    }

    public synchronized boolean retirar() {
        if (items > 0) {
            items--;
            return true;
        }
        return false;
    }

    public synchronized boolean colocar() {
        items++;
        return true;
    }
    // main igual ao da q2a
}
```

Arquivo: src/q2c/Consumidor.java

```
package q2c;
```

```
/**
```

```

* Faz 20 tentativas de consumo, esperando 'tempo' ms entre elas.
* Se o depósito estiver vazio, a tentativa falha (abordagem safety first).
*/
public class Consumidor extends Thread {
    private static final int TOTAL = 20;
    private static int contador = 0;
    private final Deposito deposito;
    private final int tempo;

    public Consumidor(Deposito deposito, int tempo) {
        super("Consumidor " + (++contador));
        this.deposito = deposito;
        this.tempo = tempo;
    }

    @Override
    public void run() {
        int consumidas = 0;
        for (int i = 1; i <= TOTAL; i++) {
            if (deposito.retirar()) {
                consumidas++;
                System.out.println(getName() + " retirou uma caixa"
                    + " | itens no deposito: " + deposito.getNumItens());
            } else {
                System.out.println(getName() + " NAO conseguiu retirar: deposito vazio");
            }
            try {
                Thread.sleep(tempo);
            } catch (InterruptedException e) {
                return;
            }
        }
        System.out.println(getName() + " terminou: consumiu " + consumidas + " de " + TOTAL +
            " caixas");
    }
}

```

```
Terminal - java q2c_run1

$ java -cp out q2c.Deposito
Execucao do main da classe Deposito terminada
Consumidor 2 NAO conseguiu retirar: deposito vazio
Consumidor 3 NAO conseguiu retirar: deposito vazio
Consumidor 5 NAO conseguiu retirar: deposito vazio
Consumidor 4 NAO conseguiu retirar: deposito vazio
Consumidor 1 retirou uma caixa | itens no deposito: 0
Produtor colocou a caixa 1 | itens no deposito: 1
Produtor colocou a caixa 2 | itens no deposito: 1
Consumidor 2 retirou uma caixa | itens no deposito: 0
Consumidor 4 NAO conseguiu retirar: deposito vazio
Produtor colocou a caixa 3 | itens no deposito: 1
Consumidor 5 retirou uma caixa | itens no deposito: 0
Consumidor 3 NAO conseguiu retirar: deposito vazio
Consumidor 1 NAO conseguiu retirar: deposito vazio
...
Execucao do main da classe Deposito terminada
Consumidor 2 terminou: consumiu 9 de 20 caixas
Consumidor 4 terminou: consumiu 13 de 20 caixas
Consumidor 1 terminou: consumiu 11 de 20 caixas
Consumidor 3 terminou: consumiu 13 de 20 caixas
Consumidor 5 terminou: consumiu 9 de 20 caixas
Produtor terminou: produziu 100 caixas
...
# ultimas linhas:
Produtor colocou a caixa 96 | itens no deposito: 41
Produtor colocou a caixa 97 | itens no deposito: 42
Produtor colocou a caixa 98 | itens no deposito: 43
Produtor colocou a caixa 99 | itens no deposito: 44
Produtor colocou a caixa 100 | itens no deposito: 45
Produtor terminou: produziu 100 caixas
```

Figura 14 – q2c – execução 1

```
Terminal - java q2c_run2

$ java -cp out q2c.Deposito
Consumidor 2 NAO conseguiu retirar: deposito vazio
Consumidor 4 NAO conseguiu retirar: deposito vazio
Consumidor 3 NAO conseguiu retirar: deposito vazio
Execucao do main da classe Deposito terminada
Consumidor 5 NAO conseguiu retirar: deposito vazio
Consumidor 1 retirou uma caixa | itens no deposito: 0
Produtor colocou a caixa 1 | itens no deposito: 1
Produtor colocou a caixa 2 | itens no deposito: 1
Consumidor 2 retirou uma caixa | itens no deposito: 0
Consumidor 4 NAO conseguiu retirar: deposito vazio
Produtor colocou a caixa 3 | itens no deposito: 1
Consumidor 5 retirou uma caixa | itens no deposito: 0
Consumidor 3 NAO conseguiu retirar: deposito vazio
Consumidor 1 NAO conseguiu retirar: deposito vazio
...
Execucao do main da classe Deposito terminada
Consumidor 4 terminou: consumiu 8 de 20 caixas
Consumidor 2 terminou: consumiu 11 de 20 caixas
Consumidor 3 terminou: consumiu 11 de 20 caixas
Consumidor 1 terminou: consumiu 11 de 20 caixas
Consumidor 5 terminou: consumiu 14 de 20 caixas
Produtor terminou: produziu 100 caixas
...
# ultimas linhas:
Produtor colocou a caixa 96 | itens no deposito: 41
Produtor colocou a caixa 97 | itens no deposito: 42
Produtor colocou a caixa 98 | itens no deposito: 43
Produtor colocou a caixa 99 | itens no deposito: 44
Produtor colocou a caixa 100 | itens no deposito: 45
Produtor terminou: produziu 100 caixas
```

Figura 15 – q2c – execução 2

2d) Última mensagem e por quê

Última mensagem: novamente "Produtor terminou: produziu 100 caixas", mas agora com o depósito terminando com 45 caixas (nas duas execuções).

- O estoque nunca ficou negativo (mínimo = 0) e não houve mais perda de atualizações: 100 produzidas – 55 consumidas = 45, valor exato — efeito do synchronized.
- Como o consumo é mais rápido que a produção, os consumidores frequentemente encontram o depósito vazio: foram 45 tentativas frustradas por execução. Cada consumidor desiste daquela tentativa (abordagem conservadora / safety first) e, ao final das 20 tentativas, terminou com apenas 8 a 14 caixas.

- Os consumidores terminam em no máximo 3 s; o produtor continua até ~5 s colocando caixas que ninguém mais vai retirar. Por isso ele é o último a imprimir e sobram 45 caixas.
- Detalhe: na execução 2 a mensagem do main apareceu apenas na 4ª linha, depois de mensagens dos consumidores — mais um exemplo de que a ordem entre threads não é garantida.

Questão 3

Descrição: quando a pré-condição de um método não é satisfeita, pode-se adotar uma abordagem otimista (liveness first — esperar), conservadora (safety first — devolver erro) ou intermediária (esperar com tempo limite). Modificar o Consumidor para que, se não conseguir retirar, espere 200 ms e tente novamente.

3a) Consumidor com nova tentativa

Explicação: a q2c implementou a abordagem conservadora (a tentativa falha e é descartada). Aqui foi implementada a abordagem otimista: enquanto retirar() retornar false, o consumidor dorme 200 ms e tenta de novo, assumindo que cedo ou tarde o produtor colocará uma caixa. Assim, cada consumidor só termina depois de efetivamente consumir 20 caixas. Deposito (synchronized) e Produtor são os mesmos da q2c.

Arquivo: src/q3/Consumidor.java

```
package q3;

/**
 * Questão 3a - Abordagem otimista (liveness first): se não conseguir retirar,
 * espera 200 ms e tenta novamente, até consumir as 20 caixas.
 */
public class Consumidor extends Thread {
    private static final int TOTAL = 20;
    private static final int ESPERA_NOVA_TENTATIVA = 200;
    private static int contador = 0;
    private final Deposito deposito;
    private final int tempo;

    public Consumidor(Deposito deposito, int tempo) {
        super("Consumidor " + (++contador));
        this.deposito = deposito;
        this.tempo = tempo;
    }

    @Override
    public void run() {
        try {
            for (int i = 1; i <= TOTAL; i++) {
                while (!deposito.retirar()) {
                    System.out.println(getName() + " encontrou o deposito vazio. Tentando de
novo em "
                                + ESPERA_NOVA_TENTATIVA + " ms");
                    Thread.sleep(ESPERA_NOVA_TENTATIVA);
                }
                System.out.println(getName() + " retirou a caixa " + i
                                + " | itens no deposito: " + deposito.getNumItens());
                Thread.sleep(tempo);
            }
        } catch (InterruptedException e) {
            return;
        }
        System.out.println(getName() + " terminou: consumiu " + TOTAL + " caixas");
    }
}
```

```
Terminal – java q3_run1

$ java -cp out q3.Deposito
Execucao do main da classe Deposito terminada
Consumidor 2 encontrou o deposito vazio. Tentando de novo em 200
ms
Consumidor 4 encontrou o deposito vazio. Tentando de novo em 200
ms
Consumidor 3 encontrou o deposito vazio. Tentando de novo em 200
ms
Consumidor 5 encontrou o deposito vazio. Tentando de novo em 200
ms
Consumidor 1 retirou a caixa 1 | itens no deposito: 0
Produtor colocou a caixa 1 | itens no deposito: 1
Produtor colocou a caixa 2 | itens no deposito: 1
Produtor colocou a caixa 3 | itens no deposito: 2
Consumidor 1 retirou a caixa 2 | itens no deposito: 1
Produtor colocou a caixa 4 | itens no deposito: 2
Consumidor 4 retirou a caixa 1 | itens no deposito: 1
Consumidor 5 retirou a caixa 1 | itens no deposito: 0
Consumidor 3 encontrou o deposito vazio. Tentando de novo em 200
ms
...
Execucao do main da classe Deposito terminada
Consumidor 5 terminou: consumiu 20 caixas
Consumidor 4 terminou: consumiu 20 caixas
Consumidor 3 terminou: consumiu 20 caixas
Produtor terminou: produziu 100 caixas
Consumidor 1 terminou: consumiu 20 caixas
Consumidor 2 terminou: consumiu 20 caixas
...
# ultimas linhas:
Produtor colocou a caixa 100 | itens no deposito: 2
Consumidor 2 retirou a caixa 19 | itens no deposito: 1
Produtor terminou: produziu 100 caixas
Consumidor 2 retirou a caixa 20 | itens no deposito: 0
Consumidor 1 terminou: consumiu 20 caixas
Consumidor 2 terminou: consumiu 20 caixas
```

Figura 16 – q3 – execução 1

```
Terminal – java q3_run2

$ java -cp out q3.Deposito
Execucao do main da classe Deposito terminada
Consumidor 5 encontrou o deposito vazio. Tentando de novo em 200
ms
Consumidor 2 encontrou o deposito vazio. Tentando de novo em 200
ms
Consumidor 3 encontrou o deposito vazio. Tentando de novo em 200
ms
Consumidor 4 encontrou o deposito vazio. Tentando de novo em 200
ms
Consumidor 1 retirou a caixa 1 | itens no deposito: 0
Produtor colocou a caixa 1 | itens no deposito: 1
Produtor colocou a caixa 2 | itens no deposito: 1
Produtor colocou a caixa 3 | itens no deposito: 2
Consumidor 1 retirou a caixa 2 | itens no deposito: 1
Produtor colocou a caixa 4 | itens no deposito: 2
Consumidor 2 retirou a caixa 1 | itens no deposito: 0
Consumidor 3 retirou a caixa 1 | itens no deposito: 1
Consumidor 5 encontrou o deposito vazio. Tentando de novo em 200
ms
...
Execucao do main da classe Deposito terminada
Consumidor 3 terminou: consumiu 20 caixas
Consumidor 1 terminou: consumiu 20 caixas
Consumidor 4 terminou: consumiu 20 caixas
Produtor terminou: produziu 100 caixas
Consumidor 5 terminou: consumiu 20 caixas
Consumidor 2 terminou: consumiu 20 caixas
...
# ultimas linhas:
Produtor colocou a caixa 100 | itens no deposito: 2
Consumidor 5 retirou a caixa 20 | itens no deposito: 1
Produtor terminou: produziu 100 caixas
Consumidor 2 retirou a caixa 20 | itens no deposito: 0
Consumidor 5 terminou: consumiu 20 caixas
Consumidor 2 terminou: consumiu 20 caixas
```

Figura 17 – q3 – execução 2

3b) Última mensagem e por quê

Última mensagem: "Consumidor 2 terminou: consumiu 20 caixas" (nas duas execuções), com o depósito terminando com 0 caixas.

- Agora todos os consumidores consomem as 20 caixas: $5 \times 20 = 100 =$ total produzido, e o estoque fecha exatamente em 0.

- Como a demanda é maior que a oferta, os consumidores passam a depender do ritmo do produtor: houve 65 e 61 esperas de 200 ms. O tempo total deixa de ser determinado pelo tempo de cada consumidor e passa a ser o do produtor (~5 s).
- O produtor termina antes dos últimos consumidores: depois de colocar a 100ª caixa, ainda é preciso que um consumidor (que pode estar em uma espera de até 200 ms ou no seu intervalo de consumo) a retire e então imprima sua mensagem de término. Por isso a última mensagem é de um consumidor.
- Risco dessa abordagem: se a quantidade total pedida pelos consumidores fosse maior que a produzida (por exemplo, 6 consumidores), os consumidores excedentes ficariam tentando para sempre — um problema de liveness (starvation). A abordagem intermediária (tentar por um tempo limite e depois desistir) evita isso.
- A espera com sleep em laço é uma espera ativa com polling: gasta tentativas e pode reagir com até 200 ms de atraso. O mecanismo idiomático em Java seria wait()/notifyAll() ou BlockingQueue.

4. Resultados e Discussão

4.1 Comparação entre execuções

Cenário	Linhas em ~2 s	Rajada média	Distribuição entre racers	Ordem previsível?
Sem sleep (q1c)	80 mil – 115 mil	43 – 65 linhas	Desigual (até 2× de diferença)	Não
Com sleep 100 ms (q1d)	160 – 186	1 linha	Igual (± 1)	Não (dentro de cada rodada)
Com prioridade (q1e)	106 mil – 148 mil	45 – 67 linhas	Desigual, sem relação com a prioridade	Não
Com join (q1f)	10.013 (fixo)	—	1000 cada	Só entre grupos (ímpares → pares)

Cenário	Estoque mínimo	Estoque final	Caixas consumidas	Última mensagem
q2a – sem controle	-43 / -44	4 / 1 (errado)	100 ("fictícias")	Produtor terminou
q2c – items > 0 + synchronized	0	45	55	Produtor terminou
q3 – espera 200 ms e re-tenta	0	0	100	Consumidor terminou

4.2 Conceitos de concorrência evidenciados

- **Não determinismo:** a mesma entrada produz saídas diferentes a cada execução (ordem de largada, vencedor, ordem de chegada, posição da mensagem do main).
- **Escalonamento preemptivo:** quem executa é decidido pelo SO; start() não significa "executar agora".
- **Recurso compartilhado e exclusão mútua:** System.out (lock interno do println) e o Depósito.
- **Condição de corrida e perda de atualização:** items = getNumItems() - 1 não é atômico (q2a).
- **Seção crítica e monitores:** synchronized tornou o teste-e-decremento atômico (q2c/q3).
- **Estados de uma thread:** RUNNABLE, TIMED_WAITING (sleep), WAITING (join) e TERMINATED.
- **Sincronização por término:** join estabelece uma ordem (relação happens-before) entre threads.
- **Safety × liveness:** q2c prioriza segurança (desiste), q3 prioriza progresso (espera), com risco de espera indefinida.

4.3 Relação com problemas reais em sistemas distribuídos

Os mesmos fenômenos aparecem, ampliados, em sistemas distribuídos, onde não há memória compartilhada nem relógio global:

- **Estoque e reservas:** o "depósito negativo" da q2a é o mesmo problema de vender mais ingressos ou produtos do que o estoque em um e-commerce quando dois servidores leem o

mesmo saldo. As soluções são transações, locks distribuídos (ex.: Redis, ZooKeeper) ou controle otimista de concorrência (versões).

- **Filas de mensagens:** Produtor/Consumidor é o modelo de sistemas como RabbitMQ e Kafka, que usam filas bloqueantes, backpressure e limite de capacidade (o atributo capacidade do Depósito).
- **Retry com espera:** a q3 corresponde ao padrão retry usado em chamadas a serviços remotos. Na prática usa-se timeout, número máximo de tentativas e backoff exponencial (abordagem intermediária) para não ficar esperando para sempre nem sobrecarregar o serviço.
- **Barreiras e coordenação:** o join equivale a esperar que todas as tarefas de uma etapa terminem antes da próxima (como as fases map e reduce do MapReduce).
- **Ordem de eventos:** assim como a ordem de impressão não é garantida entre threads, a ordem de chegada de mensagens entre máquinas também não é, o que motiva relógios lógicos (Lamport) e algoritmos de consenso.

5. Conclusão

A prática mostrou, com dados medidos, que a execução de threads é inerentemente não determinística: a ordem de início, a ordem de término e a divisão do tempo de processador mudam a cada execução. Ferramentas como sleep e setPriority apenas influenciam o escalonador — sleep de forma visível (distribuição justa e menor uso de CPU), setPriority de forma imperceptível neste ambiente — e não devem ser usadas para garantir correteza. Já join oferece uma garantia real de ordem entre threads.

No problema Produtor/Consumidor ficou evidente que o acesso concorrente a um dado compartilhado sem controle produz resultados incorretos (estoque negativo e perda de atualizações) mesmo em um programa de poucas linhas. A exclusão mútua com synchronized e a verificação de pré-condições resolvem a segurança, e a escolha entre desistir (safety first) ou esperar (liveness first) é uma decisão de projeto com consequências: perda de requisições em um caso, risco de espera indefinida no outro.

Em sistemas distribuídos esses problemas são ainda mais difíceis, pois envolvem latência de rede, falhas parciais e ausência de estado global; por isso o controle de concorrência é essencial para garantir consistência dos dados e disponibilidade do serviço.

Possíveis melhorias:

- Substituir o polling com sleep por wait()/notifyAll() no Depósito, acordando o consumidor apenas quando houver caixa, ou usar java.util.concurrent.BlockingQueue / Semaphore.
- Implementar a abordagem intermediária (tempo limite): tentar retirar por até N ms e então desistir.
- Respeitar o atributo capacidade, bloqueando o produtor quando o depósito estiver cheio.
- Usar AtomicInteger para o contador, ou ReentrantLock com Condition para maior controle.
- Usar ExecutorService em vez de criar threads manualmente e interromper os racers infinitos com interrupt().

6. Referências

1. ORACLE. Class Thread – Java Platform SE 7 API Specification. Disponível em: <http://docs.oracle.com/javase/7/docs/api/java/lang/Thread.html>.
2. ORACLE. The Java Tutorials – Concurrency. Disponível em: <https://docs.oracle.com/javase/tutorial/essential/concurrency/>.
3. TUTORIALSPPOINT. Java – Multithreading. Disponível em: http://www.tutorialspoint.com/java/java_multithreading.htm.
4. CAELUM. Apostila Java e Orientação a Objetos – Programação Concorrente e Threads. Disponível em: <http://www.caelum.com.br/apostila-java-orientacao-objetos/programacao-concorrente-e-threads/>.
5. GOETZ, B. et al. Java Concurrency in Practice. Addison-Wesley, 2006.
6. TANENBAUM, A. S.; VAN STEEN, M. Sistemas Distribuídos: princípios e paradigmas. 2. ed. Pearson, 2007.
7. COULOURIS, G. et al. Sistemas Distribuídos: conceitos e projeto. 5. ed. Bookman, 2013.

Apêndice A – Uso de ferramenta de Inteligência Artificial

Para a análise das execuções das threads e a elaboração das considerações apresentadas neste relatório, foi utilizada a ferramenta Claude Code (Anthropic), um assistente de programação baseado em inteligência artificial executado no terminal.

A ferramenta foi empregada para:

- **Processamento das saídas:** leitura dos arquivos gerados em cada execução (pasta saidas/), que chegavam a mais de 100 mil linhas nos exemplos com while(true).
- **Levantamento de métricas:** contagem de impressões por racer, ordem da primeira impressão, tamanho médio das rajadas, ordem de chegada na q1f, estoque mínimo e final do depósito e número de tentativas frustradas dos consumidores.
- **Verificações automáticas:** por exemplo, confirmar que nenhum racer par imprimiu antes do término dos ímpares (q1f) e que a soma de caixas consumidas fechava com o estoque final (q2c).
- **Interpretação dos resultados:** relacionar os dados observados aos conceitos teóricos (escalonamento, condição de corrida, exclusão mútua, safety × liveness) e aos problemas de sistemas distribuídos.
- **Instalação do ambiente:** Instalação automatizada do ambiente de programação em java, dispondo comandos de execução dos blocos de código e versionamento para o repositório.

Todos os números citados foram obtidos de execuções reais do código no ambiente descrito na Seção 2.1, e as análises foram revisadas pelo(a) autor(a) deste relatório.